PERTEMUAN KE-2
Machine Learning

Supervised Learning – Regresi dan Konsep Evaluasi Model

1. Konsep Regresi Linear Sederhana

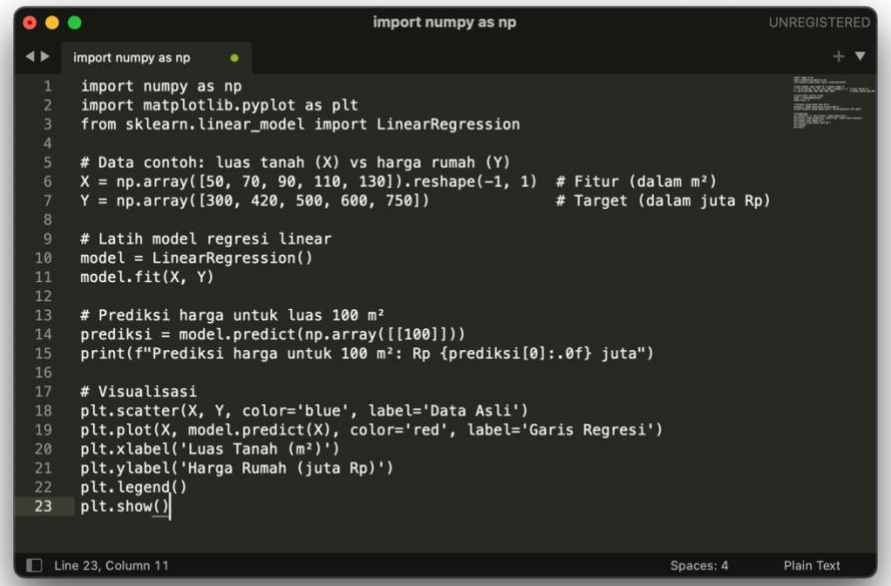
Regresi linear sederhana memodelkan hubungan antara satu variabel independen (fitur, X) dan satu variabel dependen (target, Y) dengan persamaan garis lurus:

$$Y = \beta_0 + \beta_1 X + \epsilon$$

- Y: Variabel dependen (target).
- X: Variabel independen (fitur).
- β_0: Intercept (nilai Y saat X=0).
- β_1: Koefisien kemiringan (slope).
- ϵ: Error (deviasi dari garis regresi).

Contoh :

Memprediksi harga rumah berdasarkan luas tanah (prediksi nilai *y* berdasarkan nilai *x*)



2. Regresi Linear Multivariat

Memodelkan hubungan antara beberapa variabel independen (X_1, X_2, ...) dan satu variabel dependen (Y) dengan persamaan:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \ldots + \beta_n X_n + \epsilon$$

Contoh Kasus:

Memprediksi harga rumah (Y) berdasarkan:

- Luas tanah (X_1),
- Jumlah kamar (X_2),
- Jarak ke pusat kota (X_3).

Contoh :

Prediksi harga rumah berdasarkan : Luas tanah (x_1), Jumlah kamar (x_2), Jarak stasiun (x_3)

import pandas as pd UNREGISTERED

import pandas as pd

```python
import pandas as pd
from sklearn.linear_model import LinearRegression

# Data contoh multivariat
data = {
    'Luas (m²)': [50, 70, 90, 110, 130],
    'Kamar': [2, 3, 3, 4, 4],
    'Jarak (km)': [10, 5, 3, 2, 1],
    'Harga (juta Rp)': [300, 420, 500, 600, 750]
}
df = pd.DataFrame(data)

# Pisahkan fitur (X) dan target (Y)
X = df[['Luas (m²)', 'Kamar', 'Jarak (km)']]
Y = df['Harga (juta Rp)']

# Latih model
model = LinearRegression()
model.fit(X, Y)

# Koefisien regresi
print(f"Intercept (β₀): {model.intercept_:.2f}")
print(f"Koefisien (β₁, β₂, β₃): {model.coef_}")

# Prediksi untuk rumah: 100m², 3 kamar, 4 km dari pusat kota
prediksi = model.predict([[100, 3, 4]])
print(f"Prediksi harga: Rp {prediksi[0]:.0f} juta")
```

Line 27, Column 52 Spaces: 4 Plain Text

3. Perbedaan

Aspek	Regresi Linear Sederhana	Regresi Linear Multivariat
Jumlah Variabel	1 fitur (X) + 1 target (Y)	Beberapa fitur (X_1, X_2, ...) + 1 target (Y)
Persamaan	$Y = \beta_0 + \beta_1 X$	$Y = \beta_0 + \beta_1 X_1 + ... + \beta_n X_n$
Kompleksitas	Mudah diinterpretasi	Memerlukan analisis multikolinearitas
Contoh Aplikasi	Harga rumah vs luas tanah	Harga rumah vs luas, kamar, lokasi

4. Mean Squared Error (MSE)

Konsep

- Mengukur rata-rata kuadrat selisih antara nilai prediksi (*Y*) dan nilai aktual (*Y*).
- Rumus:

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2$$

- Semakin kecil MSE, semakin baik model.
- Semakin kecil MSE, semakin baik model.

Contoh Source Code

```
from sklearn.metrics import mean_squared_error

# Data aktual (Y) dan prediksi (Y_pred)
Y = np.array([300, 420, 500, 600, 750])
Y_pred = model.predict(X)  # Misal: [320, 410, 490, 610, 740]

# Hitung MSE
mse = mean_squared_error(Y, Y_pred)
print(f"MSE: {mse:.2f}")
```

5. Root Mean Squared Error (RMSE)

Konsep

- Akar kuadrat dari MSE, memberikan error dalam skala yang sama dengan variabel target.
- Rumus:

$$\text{RMSE} = \sqrt{\text{MSE}}$$

- Interpretasi**:** RMSE = 12 berarti rata-rata error prediksi adalah ±12 unit.

Contoh Source Code

```
rmse = np.sqrt(mse)
print(f"RMSE: {rmse:.2f}")
```

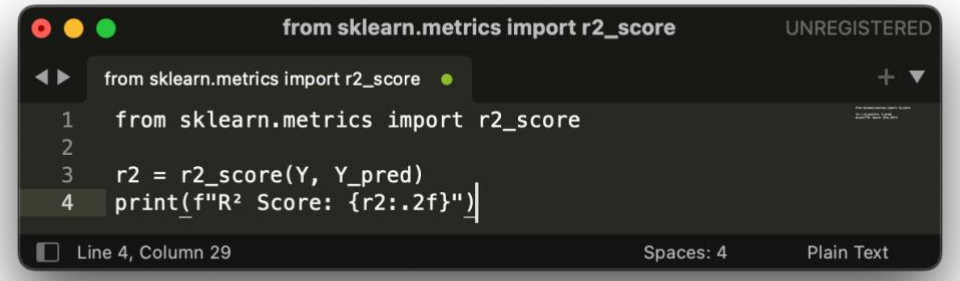
6. Koefisien Determinasi (R^2 Score)

- Mengukur proporsi variabilitas data yang dijelaskan oleh model.
- Rumus :

$$R^2 = 1 - \frac{\sum(Y_i - \hat{Y}_i)^2}{\sum(Y_i - \bar{Y})^2}$$

- Nilai :
  a. $R^2 = 1$: Model sempurna.
  b. $R^2 = 0$: Model tidak lebih baik dari rata-rata.
  c. $R^2 < 0$: Model lebih buruk dari rata-rata.

Contoh Source Code



7. Contoh Lengkap Evaluasi Model

Data

- Fitur (X): Luas tanah ([50, 70, 90, 110, 130]).
- Target (Y): Harga rumah ([300, 420, 500, 600, 750]).

Contoh Source Code

```python
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Data
X = np.array([50, 70, 90, 110, 130]).reshape(-1, 1)
Y = np.array([300, 420, 500, 600, 750])

# Train model
model = LinearRegression()
model.fit(X, Y)
Y_pred = model.predict(X)

# Evaluasi
mse = mean_squared_error(Y, Y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(Y, Y_pred)

print(f"MSE: {mse:.2f}")
print(f"RMSE: {rmse:.2f}")
print(f"R² Score: {r2:.2f}")
```

8. Perbandingan Matriks

Metrik	Formula	Interpretasi	Kelebihan	Kekurangan	Kapan Digunakan?
MSE (Mean Squared Error)	$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2$	Rata-rata kuadrat error. → Semakin kecil semakin baik.	1. Menghukum error besar lebih berat (kuadrat). 2. Mudah dihitung.	1. Sensitif terhadap outlier. 2. Satuan tidak sama dengan target (kuadrat).	Untuk fokus pada error besar (misal: prediksi risiko tinggi).
RMSE (Root Mean Squared Error)	$\text{RMSE} = \sqrt{\text{MSE}}$	Akar MSE, satuan sama dengan target. → Error rata-rata ±X unit.	1. Satuan sama dengan target. 2. Lebih mudah diinterpretasi.	Masih terpengaruh outlier.	Untuk laporan bisnis yang butuh interpretasi langsung.
R^2 (R-Squared)	$R^2 = 1 - \frac{\sum(Y_i - \hat{Y}_i)^2}{\sum(Y_i - \bar{Y})^2}$	Proporsi variasi data yang dijelaskan model (0-1). → Semakin mendekati 1 semakin baik.	1. Tidak bergantung skala data. 2. Cocok untuk bandingkan model.	1. Tidak menunjukkan besaran error. 2. Bisa menipu jika fitur tidak relevan.	Untuk memilih model terbaik pada dataset yang sama.
MAE (Mean Absolute Error)	$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}\lVert Y_i - \hat{Y}_i \rVert$	Rata-rata absolut error. → Error ±X unit (tanpa kuadrat).	1. Robust terhadap outlier. 2. Satuan sama dengan target.	Tidak menghukum error besar seberat MSE.	Jika data banyak outlier.

9. Prediksi Harga Rumah

a. Persiapan Data

Dataset dengan simulasi :

- luas_tanah (m^2)
- jumlah_kamar
- jarak_ke_pusat_kota (km)
- tahun_dibangun
- harga (target, juta Rp)

```python
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split

# Generate data acak
np.random.seed(42)
data = {
    'luas_tanah': np.random.randint(50, 200, 100),
    'jumlah_kamar': np.random.randint(1, 5, 100),
    'jarak_ke_pusat_kota': np.round(np.random.uniform(1, 20, 100), 1),
    'tahun_dibangun': np.random.randint(1990, 2023, 100),
    'harga': 0  # Akan dihitung
}
df = pd.DataFrame(data)

# Simulasikan harga berdasarkan rumus + noise
df['harga'] = (
    5 * df['luas_tanah'] +
    50 * df['jumlah_kamar'] -
    10 * df['jarak_ke_pusat_kota'] -
    0.5 * (2023 - df['tahun_dibangun']) +
    np.random.normal(0, 50, 100)
)

# Split data: 80% training, 20% testing
X = df.drop('harga', axis=1)
y = df['harga']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

b. Preprocessing Data

- Scaling untuk fitur numerik.
- Encoding untuk fitur kategorikal (jika ada).

```python
from sklearn.preprocessing import StandardScaler
from sklearn.compose import ColumnTransformer

# Definisikan transformer
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), ['luas_tanah', 'jarak_ke_pusat_kota',
        'tahun_dibangun']),
        # Jika ada kategorikal: ('cat', OneHotEncoder(), ['kategori_fitur'])
    ]
)

# Apply preprocessing
X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed = preprocessor.transform(X_test)
```

c. Membangun Model

RandomForestRegressor (handles non-linear relationships)

```python
from sklearn.ensemble import RandomForestRegressor

# Inisialisasi model
model = RandomForestRegressor(n_estimators=100, random_state=42)

# Training
model.fit(X_train_processed, y_train)
```

d. Evaluasi Model

MSE, RMSE, R^2, dan MAE.

```python
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error

# Prediksi
y_pred = model.predict(X_test_processed)

# Hitung metrik
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)

print(f"""
METRIK EVALUASI:
- MSE      : {mse:.2f}
- RMSE     : {rmse:.2f} juta Rp
- R² Score : {r2:.2f}
- MAE      : {mae:.2f} juta Rp
""")
```

e. Prediksi Data Baru

```python
# Contoh data baru
data_baru = pd.DataFrame({
    'luas_tanah': [120, 80],
    'jumlah_kamar': [3, 2],
    'jarak_ke_pusat_kota': [5, 15],
    'tahun_dibangun': [2010, 2000]
})

# Preprocess + prediksi
data_baru_processed = preprocessor.transform(data_baru)
prediksi_harga = model.predict(data_baru_processed)

print("Prediksi Harga Rumah (juta Rp):", prediksi_harga.round(2))
```

f. Visualisasi

```python
import matplotlib.pyplot as plt

# Feature importance
importance = model.feature_importances_
features = X.columns

plt.barh(features, importance)
plt.xlabel('Importance Score')
plt.title('Kontribusi Fitur terhadap Prediksi Harga')
plt.show()
```

10. Tuning

a. Feature Engineering :

- Tambahkan fitur interaksi (misal: `luas_tanah × jumlah_kamar`).
- Transformasi logaritmik untuk fitur skewed.

b. Hypermeter Tuning :

```
from sklearn.model_selection import GridSearchCV
param_grid = {'n_estimators': [50, 100, 200], 'max_depth': [None, 10, 20]}
grid_search = GridSearchCV(RandomForestRegressor(), param_grid, cv=5)
grid_search.fit(X_train_processed, y_train)
print("Best Params:", grid_search.best_params_)
```

c. Residual Plot

```
residuals = y_test - y_pred
plt.scatter(y_pred, residuals)
plt.axhline(y=0, color='r', linestyle='--')
plt.xlabel('Predicted Values')
plt.ylabel('Residuals')
plt.title('Residual Plot')
plt.show()
```

Referensi:

- *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (O'Reilly).*
- Géron, A. (2019). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (2nd ed.). O'Reilly Media.
- Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning* (2nd ed.). Springer.
- James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). *An Introduction to Statistical Learning with Applications in R*. Springer.

- Raschka, S., & Mirjalili, V. (2019). *Python Machine Learning* (3rd ed.). Packt Publishing.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.